Lab 2 – Python Basics for AI

Task 1: Python Classes and Memory Concepts

1 Instantiate a class and display namespace

```
class Sample:
    def __init__(self, value):
        self.value = value


obj = Sample(10)
print(obj.__dict__)   # Displays namespace of the instance
```

2. Function `student_data()`

```
def student_data(student_id, student_name=None, student_class=None):
    print("Student ID:", student_id)
    if student_name:
        print("Student Name:", student_name)
    if student_class:
        print("Student Class:", student_class)


student_data(101)
student_data(102, "Ali")
student_data(103, "Sara", "AI-1")
```

3. Class type, dict keys, and module

```
class Student:
    pass
```

```
print(type(Student))         # <class 'type'>
print(Student.__dict__.keys()) # Keys of dict attribute
print(Student.__module__)     # Module name
```

4.Empty classes and verification

```
class Student:
    pass

class Marks:
    pass

s = Student()
m = Marks()

print(isinstance(s, Student))   # True
print(isinstance(m, Marks))     # True
print(issubclass(Student, object)) # True
print(issubclass(Marks, object))   # True
```

5. Modify attributes

```
class Student:
    def __init__(self, name, marks):
        self.name = name
```

```
        self.marks = marks

s = Student("Ali", 85)
print("Original:", s.name, s.marks)

s.name = "Sara"
s.marks = 90
print("Updated:", s.name, s.marks)
```

6. Add and remove attributes

```
class Student:
    def __init__(self, student_id, student_name):
        self.student_id = student_id
        self.student_name = student_name

s = Student(101, "Ali")
s.student_class = "AI-1"
print(s.__dict__)

del s.student_name
print(s.__dict__)
```

7. Extend Student class

```
class Student:
    def __init__(self, student_id, student_name, student_class):
        self.student_id = student_id
        self.student_name = student_name
        self.student_class = student_class
```

```
    def display(self):
        for attr, value in self.__dict__.items():
            print(attr, ":", value)

s = Student(101, "Ali", "AI-1")
s.display()
```

8. Two instances

```
student1 = Student(101, "Ali", "AI-1")
student2 = Student(102, "Sara", "AI-2")

for i, stu in enumerate([student1, student2], start=1):
    print(f"Student {i}:")
    stu.display()
```

9. Memory allocation analysis

```
import sys

print("Memory size of student1:", sys.getsizeof(student1))
print("Memory size of student2:", sys.getsizeof(student2))
```

10. Reduce memory usage with `__slots__`

```
class StudentOptimized:
    __slots__ = ['student_id', 'student_name', 'student_class']
    def __init__(self, student_id, student_name, student_class):
        self.student_id = student_id
```

```
        self.student_name = student_name
        self.student_class = student_class

opt_student = StudentOptimized(103, "Zara", "AI-3")
print(sys.getsizeof(opt_student))
```

Task 2: Object-Oriented Design and Area Comparison

1. Classes for Trapezoid and Parallelogram

```
class Trapezoid:
    def __init__(self, a, b, h):
        self.a = a
        self.b = b
        self.h = h

    def area(self):
        return 0.5 * (self.a + self.b) * self.h

class Parallelogram:
    def __init__(self, base, height):
        self.base = base
        self.height = height

    def area(self):
```

```
        return self.base * self.height
```

2. Comparison mechanism

```
def compare_areas(shape1, shape2):
    if shape1.area() > shape2.area():
        print("First shape has larger area")
    elif shape1.area() < shape2.area():
        print("Second shape has larger area")
    else:
        print("Both shapes have equal area")
```

3. Store areas in dictionary

```
areas = {
    "Trapezoid": Trapezoid(10, 6, 5).area(),
    "Parallelogram": Parallelogram(8, 5).area()
}
print(areas)
```

4. Iterate over multiple sets

```
dimensions = [
    (Trapezoid(10, 6, 5), Parallelogram(8, 5)),
    (Trapezoid(12, 7, 6), Parallelogram(10, 6)),
    (Trapezoid(8, 5, 4), Parallelogram(6, 7))
]

for t, p in dimensions:
    print("Trapezoid area:", t.area(), "Parallelogram area:", p.area())
```

5. Maximum area

```
max_shape = max(areas, key=areas.get)

print("Shape with maximum area:", max_shape, "=", areas[max_shape])
```